UNIVERSITY OF CAPE COAST

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

COURSE CODE: INF 311

COURSE TITLE: DATA STRUCTURES

REGISTRATION NUMBER: PS/ITC/24/0398

LECTURER NAME: MISS LINDA OTOO

Describe real-world applications that require queues as a data structure, explain why queues are necessary, and choose one application to develop.

Real-World Applications That Require Queue

What is a queue?

A queue is a first-in, first-out (FIFO) data structure. Items join at the rear and leave from the front, like passengers waiting for a taxi. The main operations are enqueue (add an item to the rear), dequeue (remove and serve the front item), and peek (view the front item without removing it). A queue maintains arrival order so that the request that has waited longest is served first.

Why queues are necessary?

- **Fair ordering.** Requests can be handled in the order they arrive, reducing the chance that older requests are ignored.
- **Efficient processing.** Enqueue and dequeue can both take O(1) time when implemented with a suitable circular array or linked queue.
- **Buffering demand.** Requests can wait safely while drivers are busy, helping the system handle temporary increases in demand.
- **Separation of tasks.** Riders can submit requests independently while a dispatcher processes them when drivers become available.
- **Visibility of waiting time.** Queue position and estimated waiting time can be displayed to users and dispatchers.

Does Uber use queues?

Ride-hailing platforms need to manage incoming ride requests, driver availability, and background events. Queues are a common way to buffer and process such work. The precise internal dispatch implementation of Uber is not established by this project description, so the RideQueue application should be understood as an educational model of a possible queue-based dispatch workflow rather than a claim about Uber's exact production system.

Six real-world applications of queues (C++)

Application	How the queue is used
1. Ride-hailing dispatch (chosen application)	Ride requests enter the queue; the dispatcher serves the request at the front when assigning a driver.
2. Printers	Print jobs wait and are processed in arrival order while the printer is busy.
3. Call centres	Incoming callers wait in line until an agent becomes available.
4. Airport taxi ranks and checkout lines	Passengers or customers are served in first-in, first-out order.
5. Server request buffers	Web or API requests wait in a buffer when incoming demand exceeds immediate processing capacity.

6. Task scheduling and breadth-first search	Tasks or graph nodes can be processed in the order they are added.

Chosen application: RideQueue dispatch ride system.

RideQueue demonstrates the main purpose of a queue: serving requests in arrival order. It also makes important queue operations visible through a dispatch board, including adding requests, dispatching the next rider, viewing the next rider, cancelling a booking, and tracking queue positions.

Data design

The application uses a circular array with a fixed capacity of eight ride requests. The queue maintains a front index, a rear index, and a count of active requests. Each request can store a ride ID, rider name, and other trip details such as destination or estimated arrival time (ETA).

How the RideQueue application works

- Request Ride: adds a new ride request to the rear of the circular queue. If all eight slots are occupied, the application reports that the queue is full.
- Dispatch: removes the oldest waiting request from the front and assigns it to a driver.
- Peek Next: displays the next request to be served without removing it.
- Cancel Booking: removes a selected booking. In a straightforward array-based implementation, later items may need to shift, so cancellation takes O(n) time.
- Queue display: shows active positions such as #1 (front), #2, and so on, together with ride details and ETAs.
- Status indicators: show the front and rear indices, current queue size, total served rides, and the C++ operation associated with each action.

Circular array vs linked queue

Implementation	Advantages	Limitations
Circular array	Fixed memory, no per-request allocation, and efficient wraparound reuse of freed slots.	Capacity is fixed; overflow must be handled.
Linked queue	Can grow dynamically as requests arrive.	Requires node allocation and pointer management.

Conclusion

Queues are useful wherever requests or tasks must wait and be processed in a defined order. Examples include ride dispatch, printers, call centres, checkout lines, server buffers, and task scheduling. RideQueue illustrates how a circular array can implement a fixed-capacity FIFO queue. Enqueue and dequeue are O(1), while cancellation in the simple shifting implementation is O(n). The circular design reuses slots freed at the front, preventing the queue from running out of array space merely because the rear index has reached the last physical slot. The project therefore demonstrates both the practical value of queues and the trade-offs of different implementations.